

Table of Contents

1. Product Overview	2
2.2 Backend / Server	3
3. Architecture and Repository Structure	4
4. Prerequisites and Brand-New Mac Setup	5
5. Git Repository Setup	5
6. Database Setup (SQLite, zero external services)	6
7. Redis Policy (explicitly not required)	6
8.1 Secret handling rules (enforced)	7
9. AI Providers	8
10. Email System	9
11. Payments (Stripe and Razorpay)	9
12.1 Admin authorization	10
13. Application Startup (frontend and backend together)	10
14. Workflows and Background Jobs	11
15. Local End-to-End Test Checklist	12
16. Local to Staging to Production	12
17. Troubleshooting	13
18. Complete Command Reference	14
19. Security Notes	14

Fact	Verified value	Evidence
Framework	Next.js 16.1.3, App Router	package.json, src/app/layout.tsx
React / TypeScript	React 19.2.3 / TypeScript 5.9.3	lockfile, tsconfig.json
UI system	Tailwind CSS 4.1.18 + shadcn/ui + 27 Radix packages	globals.css, components.json
Backend API	507 Next.js Route Handlers under src/app/api/	file count
Database	SQLite file db/custom.db via Prisma 6.19.2 (105 models)	prisma/schema.prisma
Redis	Not required (optional pub/sub only)	src/lib/redis-pubsub-service.ts
AI	Built-in Z-AI SDK; optional OpenAI / Anthropic / OpenRouter / local fallbacks	src/lib/ai/ai-provider.ts
Auth	Custom JWT stack: OTP email, magic link, Google OAuth, TOTP MFA	src/lib/auth.ts
Ports	App 3000; optional realtime sidecar 3003	package.json, Makefile

Verified core facts and their sources.

1. Product Overview

Equally important is what a developer does **not** need locally: PostgreSQL, Redis, Docker, the Python FastAPI service under backend/, Celery, and the mini-services sidecars are all optional. The complete application runs on Node.js plus a SQLite file, which makes the zero-to-running path short and reproducible on a brand-new machine.

Technology	Version	Purpose
Next.js (App Router)	16.1.3	Full-stack framework; dev on Turbopack, build on webpack
React / React DOM	19.2.3	UI runtime
TypeScript	5.9.3	Strict mode (noImplicitAny relaxed)
Tailwind CSS	4.1.18	CSS-first config (globals.css @theme inline)
shadcn/ui + Radix UI	27 Radix packages	Component primitives in src/components/ui/
lucide-react	0.525.0	Icon set (tree-shaken)
framer-motion	12.26.2	Animation
recharts	2.15.4	Charts via ui/chart.tsx
zustand	5.0.10	Six client stores in src/lib/*-store.ts

Technology	Version	Purpose
@tanstack/react-query	5.90.19	Server-state cache
@dnd-kit (core/sortable/utilities)	6.3.1 / 10.0.0 / 3.2.2	Kanban drag and drop
cmdk, input-otp, sonner, react-day-picker, qrcode.react, react-markdown	various	Palette, OTP input, toasts, calendar, QR, markdown
Fonts	Geist + Geist Mono via next/font/google	src/app/layout.tsx

Frontend stack (versions resolved from package.json and lockfile).

2.2 Backend / Server

- **Runtime model:** one Next.js process serves pages, 507 API route handlers, SSE streams (/api/events/*) and server-side services; server components and route handlers share the Prisma client from src/lib/db.ts.
- **Auth libraries:** jsonwebtoken 9.0.3 (Node) and jose 6.2.3 (Edge proxy), bcryptjs 3.0.3 for password hashing (12 rounds); next-auth 4.24.11 is declared but never imported.
- **Payments:** stripe 22.1.1 (API 2025-04-30.basil) and razorpay 2.9.6.
- **Email:** resend 6.12.3 (primary) and nodemailer 8.0.7 (SMTP fallback).
- **Observability:** custom logger, metrics collector, alert engine and health checks under src/lib/observability/; OpenTelemetry packages are installed but never initialized; Sentry is a console shim behind NEXT_PUBLIC_SENTRY_DSN.
- **Validation:** zod 4.x in src/lib/security/input-validator.ts.

Path	Contents
src/app/	App Router pages + 507 API route handlers (the backend)
src/components/	249 tsx: ui/ (shadcn), dashboard/, admin/, meetings/, feedback/
src/lib/	261 ts files: ai/, security/, observability/, payments/, gmail-*, workflow-*, meetings/ and more
src/hooks/	23 client hooks (auth, SSE, websocket, payment)
src/proxy.ts	Next 16 edge proxy: route protection, CSRF, security headers (jose jwtVerify)
prisma/	schema.prisma (105 models), schema.production.prisma (PG target), 4 migrations
db/	custom.db - the live SQLite database (about 4.2 MB)
scripts/	120+ ops/dev scripts incl. backup/, seed.ts, security-scan.ts
mini-services/	5 optional Bun sidecars: realtime-service (:3003), ws-service, proxy, email-service, server-watchdog

Path	Contents
backend/	Optional Python FastAPI + Celery stack (not imported by Next.js)
docs/, infra/, deploy/	Deployment runbooks, ops strategy, platform assets
tests/	37 vitest files: unit/, integration/, e2e/, load/

Repository map (depth 2, excluding node_modules and .next).

Optional realtime sidecar: mini-services/realtime-service (Bun + Socket.IO on port 3003). The in-app primary realtime path is SSE and works without it; the client socket hook degrades gracefully with reconnection disabled.

3. Architecture and Repository Structure

Software	Version	Why needed
macOS	13 or newer	Node 20 support
Xcode Command Line Tools	latest	Provides Git and compilers
Homebrew	latest	Package manager
Node.js	20 LTS or newer (≥ 20.9)	Next.js 16 requirement; Dockerfile pins node:20-alpine
npm	10.x (bundled)	Matches vercel.json install command
Bun (optional)	1.1+	Only for scripts/seed.ts, mini-services, make targets
VS Code (optional)	latest	Editor

Required software. PostgreSQL, Redis, Docker and Python are NOT required locally.

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install node@20
brew link node@20
node -v
npm -v
```

4. Prerequisites and Brand-New Mac Setup

Not required locally, verified: PostgreSQL (the database is SQLite), Redis (optional pub/sub only), Docker (only for containerized deployment), and Python (the backend/ FastAPI stack is a separate optional service, not imported by the Next.js app).

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git clone https://github.com/Hugs-4-Bugs/AQ-OS.git AcquisitionOS
cd AcquisitionOS
git checkout main
git branch --show-current
git rev-parse HEAD
git status --porcelain
```

5. Git Repository Setup

Verification expectations: branch prints main; rev-parse prints the current commit hash (record it); status prints nothing on a clean tree. Then install dependencies with the exact command the repository expects (npm with legacy peer deps, because next-auth 4 and nodemailer 8 declare conflicting peer ranges - the same combination used by vercel.json and the Dockerfile):

```
npm install --legacy-peer-deps
```

The postinstall hook runs prisma generate automatically, so the typed Prisma client is ready before the first build. bun install also works (a bun.lock exists and the Makefile uses it), but npm is the documented default because production uses npm.

```
npm run db:generate      # generate Prisma Client (also runs via postinstall)
npm run db:push          # push schema to the SQLite file (the real dev workflow)
npx prisma studio        # browse data on http://localhost:5555
bun run scripts/seed.ts  # OPTIONAL sample data (destructive: deletes leads/deals first)
```

Operation	Command	Notes
Start / stop database	nothing to start	It is a file: db/custom.db
Schema change	npm run db:push	Canonical workflow; only 4 migrations exist vs 105 models
Formal migrations	npm run db:migrate	Works, but migration state has drifted from schema
Backup	npm run backup	Pass DB_PATH=db/custom.db (script default points elsewhere)

Operation	Command	Notes
Destructive reset	npm run db:reset	DEV ONLY - deletes all data; never run against shared data

Daily database operations (Prisma 6.19.2).

6. Database Setup (SQLite, zero external services)

Feature	Behaviour without Redis
Cross-process realtime fan-out	Works in-process; fan-out across multiple instances is lost (irrelevant locally)
Health endpoint Redis component	Reports "Redis not configured - skipped" (healthy, not failing)
Sessions, caching, rate limiting, queues	Unaffected - in-memory implementations

Everything else degrades to nothing: local development is identical without Redis.

Do not install Redis on the Mac unless you later run the optional Socket.IO sidecar, which accepts REDIS_URL for cross-instance fan-out.

7. Redis Policy (explicitly not required)

Variable	Class	Purpose / source
DATABASE_URL	required, secret	SQLite file locally; managed PostgreSQL in production
JWT_SECRET / AUTH_SECRET / NEXTAUTH_SECRET	required, secret	JWT signing; self-generate 64-hex random
NEXT_PUBLIC_APP_URL	required, client	Public URL fallback (dynamic origin wins)
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	optional, secret	Google sign-in; Google Cloud Console OAuth client
SMTP_USER / SMTP_PASSWORD	optional, secret	Gmail SMTP; 16-char Google Account App Password
RESEND_API_KEY	optional, secret	Primary email provider (resend.com API keys)
GOOGLE_SEARCH_API_KEY / GOOGLE_SEARCH_CX	optional, secret	Lead discovery search (Cloud API key + engine id)

Variable	Class	Purpose / source
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET / price IDs	optional, secret	Billing; Stripe Dashboard Developers section
RAZORPAY_KEY_ID / KEY_SECRET / WEBHOOK_SECRET	optional, secret	India gateway; Razorpay Dashboard
OPENAI / ANTHROPIC / OPENROUTER keys	optional, secret	AI fallback chain (built-in Z-AI needs no key)
CRON_SECRET	optional, secret	Bearer token for /api/cron/* endpoints; self-generate
ENCRYPTION_KEY / GMAIL_ENCRYPTION_KEY	optional, secret	Token encryption at rest; ENCRYPTION_KEY throws in production if unset
REDIS_URL	optional	Pub/sub fan-out only; omit locally

Core variables. Full 201-name inventory with sources: ENVIRONMENT-VARIABLES.md.

8.1 Secret handling rules (enforced)

- .gitignore line 34 covers .env*; .env, .env.local and .env.*.local are never committed.
- .env.example contains placeholders only and is the shareable template.
- No real credential value appears in any documentation or PDF in this repository.
- **Known leak requiring rotation (verified):** the git-tracked file ensure-env.sh embeds real-looking Google OAuth, Gmail app-password and search API values. Treat them as exposed: rotate them in Google Cloud Console / Google Account, then keep the new values only in .env.
- AUTH_DEV_MODE: when NODE_ENV is not production AND this is not the literal string "false", unsendable OTP codes are returned in API responses (src/lib/dev-auth.ts). With real email configured the flag is irrelevant; keep it false in any shared environment.

```
cp .env.example .env
# generate strong secrets (repeat for each secret name):
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# then append to .env:
#   JWT_SECRET=   AUTH_SECRET=   NEXTAUTH_SECRET=
#   CRON_SECRET=
git check-ignore -v .env    # must confirm the file is ignored
```

Provider	Trigger env vars	Default model	If missing
Z-AI (built-in)	none	SDK server-side default	Chain reports failure; credits refunded
OpenAI	OPENAI_API_KEY (+_MODEL, _BASE_URL)	gpt-4o	Not appended to chain; nothing breaks
Anthropic	ANTHROPIC_API_KEY (+_MODEL, _BASE_URL)	claude-sonnet-4-20250514	Not appended; nothing breaks
OpenRouter	OPENROUTER_API_KEY (+_MODEL, _BASE_URL)	openai/gpt-4o	Not appended; nothing breaks
Local LLM	AI_LOCAL_ENDPOINT (+AI_LOCAL_MODEL)	local-default	Not appended; nothing breaks

Provider selection and degradation (verified in ai-provider.ts).

9. AI Providers

Web research (lead discovery, deep research, website intelligence) uses Google Custom Search (GOOGLE_SEARCH_API_KEY + GOOGLE_SEARCH_CX or GOOGLE_SEARCH_ENGINE_ID) with a SerpAPI fallback (SERPAPI_KEY) and the Z-AI web_search / page_reader tools. Lead discovery fails fast with an explicit "No search API configured" error when neither search provider is set - this is the one AI-adjacent feature that hard-requires keys. Embeddings are novel: a 64-dimension vector is elicited from the chat model and stored as JSON in Prisma, with cosine similarity computed in JavaScript; no pgvector or embedding API exists.

Trigger	Route / service	Recipient
OTP login code (6 digits, 10 min, 5 attempts)	POST /api/auth/otp/request to sendEmail()	User inbox
Magic link (32-byte token, 15 min)	POST /api/auth/magic-link/request	User inbox
Suspicious-login security alert	auth.ts sendSecurityAlertEmail()	User inbox
Invoices and failed-payment notices	payment flows	User inbox
Notifications and sequence emails	notification-engine / sequence engine	User or leads

Verified trigger-to-recipient map. Provider chain: Resend, then Gmail SMTP.

10. Email System

Local behaviour with real credentials: Gmail SMTP needs a 16-character App Password from Google Account (Security, 2-Step Verification, App passwords), with SMTP_HOST smtp.gmail.com and SMTP_PORT 587. Once SMTP or Resend is configured, the login screen stops showing the "Dev mode - email delivery not configured" banner and codes are emailed for real. In production builds the dev-mode code exposure is impossible regardless of configuration.

Aspect	Stripe	Razorpay
Keys	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY, 16 price-ID names resolved	RAZORPAY_KEY_ID/KEY_SECRET/WEB HOOK_SECRET, NEXT_PUBLIC_RAZORPAY_KEY_ID, RAZORPAY_PLAN_*
Webhook endpoint	/api/payments/webhook/stripe	/api/payments/webhook/razorpay
Events handled	11 types incl. checkout.session.completed, invoice.payment_failed, charge.refunded, subscription events	8 types incl. payment.captured, subscription.charged, refund.processed
Verification	constructEvent() signature; production rejects unsigned webhooks	HMAC-SHA256 constant-time compare; webhook HMAC over raw body
Local testing	stripe listen --forward-to localhost:3000/api/payments/webhook/stripe	Test keys + dashboard-forwarded webhook

Gateway comparison (verified in the webhook routes and provider services).

11. Payments (Stripe and Razorpay)

For local Stripe testing: install the Stripe CLI (brew install stripe/stripe-cli), run stripe login, then stripe listen --forward-to localhost:3000/api/payments/webhook/stripe and copy the printed whsec value into STRIPE_WEBHOOK_SECRET. Test cards such as 4242 4242 4242 4242 complete checkout. Razorpay test keys come from the Razorpay dashboard and its webhook can be verified by triggering a test payment from the dashboard.

Flow	Sequence (verified)
OTP login	/api/auth/otp/request (email code) then /api/auth/otp/verify (session + cookies)
Magic link	/api/auth/magic-link/request then /magic-link/verify (redirect with session)
Google OAuth	/api/auth/google/state builds authUrl with dynamically derived redirect_uri, then /api/auth/callback/google exchanges the code, upserts the user (role owner, plan free, 14-day trial) and sets cookies
Session refresh	POST /api/auth/refresh rotates the pair and revokes the old session
Signout	POST /api/auth/signout revokes the session (optionally all devices)

Primary auth flows. First-login via Google creates role owner with a 14-day trial.

12.1 Admin authorization

- Roles are super_admin, owner, admin, member, viewer; every /api/admin/* handler wraps in withSuperAdmin, which demands exactly role super_admin from the database.
- The four /admin pages (overview, users, billing, feedback) are guarded server-side by getSuperAdminSession(), which re-loads the user from the database on every request.
- There is no promotion API and no SUPER_ADMIN_EMAIL variable: becoming admin is a direct DB role update (for example via Prisma Studio), documented in ADMIN-DASHBOARD-ACCESS.md.
- The documented super-admin account contact@prabhat.online logs in through the same public OTP or Google flows. No bypass exists and none should be created.

```
npm run dev          # Turbopack dev server on http://localhost:3000, logs to dev.log
curl -s localhost:3000/api/health
open http://localhost:3000
# stop: Ctrl-C (or: lsof -ti :3000 | xargs kill)
```

13. Application Startup (frontend and backend together)

Expected output starts with the Next.js 16 banner (Turbopack) and "Ready". The health endpoint returns a JSON status object. For a production-shaped run on the Mac, build with webpack (the repository already pins --webpack in both build and vercel-build scripts to avoid Turbopack memory pressure on smaller machines) and start the standalone server:

```
npm run build        # prisma generate, next build --webpack, clean-standalone
npm start            # start.js assembles .next/standalone and serves on :3000
```

If the dev server is ever killed with exit code 137 on a small-RAM Mac, that is Turbopack memory pressure, not a code error: free memory or reduce concurrent work; production builds already use webpack.

Cron endpoint (/api/cron/)	Purpose	Local trigger
expire-api-keys	Mark expired API keys	curl POST + Bearer CRON_SECRET
renew-subscriptions	Renew credit allocations; downgrades; free reset	same
credit-renewal / end-of-period	Subscription period processing	same
payment-reconciliation	Sweep Stripe/Razorpay for missed fulfillments	same (GET)
process-sequences / sequence-processing	Advance outreach sequences	same
process-gmail-replies	Classify new Gmail replies	same
hot-lead-scan / sdr-cycle	Hot lead detection; SDR autonomous cycle	same
autonomous-outreach	Process autonomous outreach queue	same
meeting-reminders	Send due meeting reminder emails	same

All 12 cron endpoints are Bearer-protected by CRON_SECRET.

14. Workflows and Background Jobs

No local scheduler is wired: trigger jobs manually with curl using Authorization: Bearer \$CRON_SECRET. In production an external scheduler (Cloudflare Worker dispatcher or GCP Cloud Scheduler, both documented in docs/deployment/) POSTs each endpoint; vercel.json contains no crons key, so Vercel Cron will not fire them automatically. Scheduled workflow triggers additionally require the optional Python Celery stack.

Area	What to verify	Expected result
Application	Homepage, /auth/signin, OTP request, verify, dashboard, signout	Pages render; real OTP email arrives; session persists
Database	/api/health, /api/health/database, Prisma Studio, CRUD in a tab	OK status; rows visible and editable
Email	OTP and magic-link requests	Real delivery via SMTP/Resend; no dev-mode banner
AI	Dashboard AI chat, lead analysis	Replies generated (Z-AI needs no key)

Area	What to verify	Expected result
Lead discovery	Discovery job with search keys configured	Leads found; without keys, explicit config error
Credits	AI actions consume credits; billing page ledger	Balance decreases per action cost
Payments	Stripe test checkout + webhook event; Razorpay test payment	Order verified; subscription/credits applied
Admin	super_admin account login, /admin sections	Overview, Users, Billing, Feedback all load
Security	Normal user opens /admin; unauthenticated API call	Redirect to /; 401/403 JSON responses

End-to-end verification matrix.

15. Local End-to-End Test Checklist

Aspect	Local	Staging	Production
Runtime	npm run dev (Turbopack)	npm run build && npm start or platform build	same as staging, live secrets
Database	SQLite file	managed PostgreSQL recommended	PostgreSQL required on Vercel
Auth secrets	self-generated dev values	separate staging secrets	separate production secrets
Google OAuth	localhost redirect whitelisted	staging redirect whitelisted	production redirect whitelisted
Payments	sk_test_ / rzp_test_	test keys + staging webhook	live keys + production webhook
Cron	manual curl	external scheduler to staging URL	external scheduler to production URL

Environment matrix (details in LOCAL-TO-PRODUCTION-DEPLOYMENT.md).

16. Local to Staging to Production

- Database migration: schema.production.prisma (PostgreSQL/Supabase) plus the guided script scripts/migrate-to-postgresql.sh and the rollback runbook infra/db-migration-rollback.md.
- Deployment targets verified in-repo: vercel.json (npm install --legacy-peer-deps, build command npm run build, region sin1; note: no crons key), root Dockerfile (node:20-alpine two-stage, standalone output, HEALTHCHECK on /api/health), and per-platform runbooks under docs/deployment/ (Cloudflare, GCP, AWS, Azure, EC2/K8s/Railway/Render).

- Dockerfile note: it copies .npmrc, which is currently absent from the repository - create an empty .npmrc or adjust the COPY line before containerized builds.
- Production webhooks to register: /api/payments/webhook/stripe, /api/payments/webhook/razorpay, optionally /api/gmail/pubsub/webhook, /api/telegram/webhook and /api/calendar/webhook.
- Backups: scripts/backup/backup.sh (sqlite3 .backup, gzip, integrity check, optional S3, 30-day rotation) - always pass DB_PATH=db/custom.db because the built-in default points elsewhere; pg_dump supported for PostgreSQL.

Symptom	Cause (verified)	Fix
npm install ERESOLVE peer conflict	next-auth 4 vs nodemailer 8 ranges	npm install --legacy-peer-deps (exact repo command)
Dev server killed, exit 137	Turbopack memory pressure on small-RAM machines	Free RAM; builds already use --webpack
Port 3000 in use	Stale process	lsof -ti :3000 xargs kill
Cannot find module @prisma/client	generate skipped	npm run db:generate
SQLITE_BUSY / database locked	Concurrent writers on the SQLite file	Retry; keep a single app instance locally
OTP shows dev-mode banner	No email provider configured	Set real SMTP_* values (or RESEND_API_KEY)
Gmail SMTP 535 auth error	Wrong/missing App Password or 2FA off	Create a 16-char App Password
Google redirect_uri_mismatch	Localhost redirect not whitelisted	Add http://localhost:3000/api/auth/callback/google
401 on APIs after login	Access cookie expired (15 min)	Re-login or POST /api/auth/refresh
403 on /api/admin/*	Account is not super_admin in DB	Update User.role (Prisma Studio)
Discovery error "No search API configured"	Missing discovery keys	Set GOOGLE_SEARCH_API_KEY + GOOGLE_SEARCH_CX or SERPAPI_KEY
Build killed (exit 137)	Build memory limit	NODE_OPTIONS=--max-old-space-size=2560 npm run build
Docker build fails at COPY .npmrc	.npmrc absent from repo	Create an empty .npmrc or remove the COPY line

Verified troubleshooting matrix.

17. Troubleshooting

18. Complete Command Reference

19. Security Notes